

Datenbankarchitektur für Buchtausch-Apps

Ein Leitfaden zur Erstellung und Implementierung einer
umfassenden Datenbanklösung

Buchtausch-App Datenbank

Diese Dokumentation beschreibt die Struktur und Implementierung einer vollständigen Datenbank für eine Buchtausch-App. Sie umfasst das Erstellen der Datenbank, das Anlegen von Tabellen, das Einfügen von Dummy-Daten, das Erstellen von Indizes und das Durchführen von Testabfragen.

1. Datenbank Erstellung

```
DROP DATABASE IF EXISTS buchtausch;  
CREATE DATABASE buchtausch CHARACTER SET utf8mb4 COLLATE  
utf8mb4_unicode_ci;  
USE buchtausch;
```

2. Tabellenstruktur

Tabelle: Nutzer

Diese Tabelle speichert Informationen über die Nutzer der App.

```
CREATE TABLE nutzer (  
  nutzer_id INT PRIMARY KEY AUTO_INCREMENT,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  passwort_hash VARCHAR(255) NOT NULL,  
  vorname VARCHAR(50) NOT NULL,  
  nachname VARCHAR(50) NOT NULL,  
  strasse VARCHAR(100) NOT NULL,  
  plz VARCHAR(10) NOT NULL,  
  stadt VARCHAR(50) NOT NULL,  
  telefon VARCHAR(20),  
  registrierungsdatum DATE NOT NULL DEFAULT CURRENT_DATE,  
  profilbild_url VARCHAR(500),  
  ist_aktiv BOOLEAN DEFAULT TRUE
```

);

Tabelle: Verlag

```
CREATE TABLE verlag (  
    verlag_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL UNIQUE,  
    ort VARCHAR(100) NOT NULL,  
    website VARCHAR(200),  
    email VARCHAR(100),  
    telefon VARCHAR(20),  
    gruendungsjahr INT  
);
```

Tabelle: Autor

```
CREATE TABLE autor (  
    autor_id INT PRIMARY KEY AUTO_INCREMENT,  
    vorname VARCHAR(50) NOT NULL,  
    nachname VARCHAR(50) NOT NULL,  
    geburtsdatum DATE,  
    nationalitaet VARCHAR(50),  
    biografie TEXT,  
    website VARCHAR(200)  
);
```

Tabelle: Genre

```
CREATE TABLE genre (  
    genre_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(50) NOT NULL UNIQUE,  
    beschreibung VARCHAR(200),  
    uebergeordnetes_genre_id INT,  
    FOREIGN KEY (uebergeordnetes_genre_id) REFERENCES genre(genre_id)  
);
```

Tabelle: Buch

```
CREATE TABLE buch (  
    buch_id INT PRIMARY KEY AUTO_INCREMENT,  
    isbn VARCHAR(13) UNIQUE,  
    titel VARCHAR(200) NOT NULL,  
    undertitel VARCHAR(200),
```

```
verlag_id INT NOT NULL,  
erscheinungsjahr INT,  
sprache VARCHAR(30) NOT NULL DEFAULT 'Deutsch',  
seitenzahl INT,  
beschreibung TEXT,  
cover_url VARCHAR(500),  
FOREIGN KEY (verlag_id) REFERENCES verlag(verlag_id)  
);
```

Tabelle: Buch_Autor (M:N Beziehung)

```
CREATE TABLE buch_autor (  
    buch_id INT NOT NULL,  
    autor_id INT NOT NULL,  
    hauptautor BOOLEAN DEFAULT FALSE,  
    PRIMARY KEY (buch_id, autor_id),  
    FOREIGN KEY (buch_id) REFERENCES buch(buch_id) ON DELETE CASCADE,  
    FOREIGN KEY (autor_id) REFERENCES autor(autor_id) ON DELETE CASCADE  
);
```

Tabelle: Buch_Genre (M:N Beziehung)

```
CREATE TABLE buch_genre (  
    buch_id INT NOT NULL,  
    genre_id INT NOT NULL,  
    PRIMARY KEY (buch_id, genre_id),  
    FOREIGN KEY (buch_id) REFERENCES buch(buch_id) ON DELETE CASCADE,  
    FOREIGN KEY (genre_id) REFERENCES genre(genre_id) ON DELETE CASCADE  
);
```

Tabelle: Standort

```
CREATE TABLE standort (  
    standort_id INT PRIMARY KEY AUTO_INCREMENT,  
    nutzer_id INT NOT NULL,  
    bezeichnung VARCHAR(50) NOT NULL,  
    strasse VARCHAR(100) NOT NULL,  
    plz VARCHAR(10) NOT NULL,  
    stadt VARCHAR(50) NOT NULL,  
    latitude DECIMAL(10, 8),  
    longitude DECIMAL(11, 8),  
    abholzeiten VARCHAR(200),
```

```
versand_option BOOLEAN DEFAULT FALSE,  
ist_standard BOOLEAN DEFAULT FALSE,  
FOREIGN KEY (nutzer_id) REFERENCES nutzer(nutzer_id) ON DELETE  
CASCADE  
);
```

Tabelle: Buch_Exemplar

```
CREATE TABLE buch_exemplar (  
    exemplar_id INT PRIMARY KEY AUTO_INCREMENT,  
    buch_id INT NOT NULL,  
    besitzer_id INT NOT NULL,  
    standort_id INT NOT NULL,  
    zustand VARCHAR(20) NOT NULL,  
    status VARCHAR(20) NOT NULL DEFAULT 'verfügbar',  
    max_leihdauer_tage INT NOT NULL DEFAULT 30,  
    notizen TEXT,  
    erstellungsdatum DATE NOT NULL DEFAULT CURRENT_DATE,  
    anzahl_ausleihen INT DEFAULT 0,  
    FOREIGN KEY (buch_id) REFERENCES buch(buch_id) ON DELETE CASCADE,  
    FOREIGN KEY (besitzer_id) REFERENCES nutzer(nutzer_id) ON DELETE  
CASCADE,  
    FOREIGN KEY (standort_id) REFERENCES standort(standort_id)  
);
```

Tabelle: Ausleihe

```
CREATE TABLE ausleihe (  
    ausleihe_id INT PRIMARY KEY AUTO_INCREMENT,  
    exemplar_id INT NOT NULL,  
    verleiher_id INT NOT NULL,  
    entleiher_id INT NOT NULL,  
    standort_id INT NOT NULL,  
    startdatum DATE NOT NULL,  
    enddatum DATE NOT NULL,  
    rueckgabedatum DATE,  
    status VARCHAR(20) NOT NULL DEFAULT 'angefragt',  
    notizen TEXT,  
    erstellungsdatum DATE NOT NULL DEFAULT CURRENT_DATE,  
    FOREIGN KEY (exemplar_id) REFERENCES buch_exemplar(exemplar_id),  
    FOREIGN KEY (verleiher_id) REFERENCES nutzer(nutzer_id),  
    FOREIGN KEY (entleiher_id) REFERENCES nutzer(nutzer_id),
```

```
FOREIGN KEY (standort_id) REFERENCES standort(standort_id)
);
```

Tabelle: Ausleihe_Anfrage (Ternär)

```
CREATE TABLE ausleihe_anfrage (
  anfrage_id INT PRIMARY KEY AUTO_INCREMENT,
  nutzer_id INT NOT NULL,
  exemplar_id INT NOT NULL,
  gewuenschter_start DATE NOT NULL,
  gewuensches_ende DATE NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'offen',
  nachricht TEXT,
  anfragedatum DATE NOT NULL DEFAULT CURRENT_DATE,
  FOREIGN KEY (nutzer_id) REFERENCES nutzer(nutzer_id),
  FOREIGN KEY (exemplar_id) REFERENCES buch_exemplar(exemplar_id)
);
```

Tabelle: Bewertung

```
CREATE TABLE bewertung (
  bewertung_id INT PRIMARY KEY AUTO_INCREMENT,
  bewertende_id INT NOT NULL,
  bewertete_id INT,
  exemplar_id INT,
  ausleihe_id INT,
  sterne INT NOT NULL,
  kommentar TEXT,
  datum DATE NOT NULL DEFAULT CURRENT_DATE,
  FOREIGN KEY (bewertende_id) REFERENCES nutzer(nutzer_id),
  FOREIGN KEY (bewertete_id) REFERENCES nutzer(nutzer_id),
  FOREIGN KEY (exemplar_id) REFERENCES buch_exemplar(exemplar_id),
  FOREIGN KEY (ausleihe_id) REFERENCES ausleihe(ausleihe_id)
);
```

Tabelle: Nachricht

```
CREATE TABLE nachricht (
  nachricht_id INT PRIMARY KEY AUTO_INCREMENT,
  sender_id INT NOT NULL,
  empfaenger_id INT NOT NULL,
  betreff VARCHAR(100) NOT NULL,
```

```
inhalt TEXT NOT NULL,  
zeitstempel DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
gelesen BOOLEAN DEFAULT FALSE,  
FOREIGN KEY (sender_id) REFERENCES nutzer(nutzer_id),  
FOREIGN KEY (empfaenger_id) REFERENCES nutzer(nutzer_id)  
);
```

Tabelle: Buch_Suche (Ternär)

```
CREATE TABLE buch_suche (  
  suche_id INT PRIMARY KEY AUTO_INCREMENT,  
  nutzer_id INT NOT NULL,  
  standort_id INT,  
  genre_id INT,  
  suchradius_km INT,  
  suchbegriff VARCHAR(200),  
  suchzeitpunkt DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  ergebnis_anzahl INT,  
  FOREIGN KEY (nutzer_id) REFERENCES nutzer(nutzer_id),  
  FOREIGN KEY (standort_id) REFERENCES standort(standort_id),  
  FOREIGN KEY (genre_id) REFERENCES genre(genre_id)  
);
```

3. Dummy-Daten

Beispiel für Dummy-Daten in der Tabelle "Nutzer":

```
INSERT INTO nutzer (email, passwort_hash, vorname, nachname, strasse, plz,  
stadt, telefon) VALUES  
( 'anna.schmidt@email.de', 'hash123', 'Anna', 'Schmidt', 'Hauptstraße 12', '10115',  
'Berlin', '01761234567'),  
( 'ben.wagner@email.de', 'hash456', 'Ben', 'Wagner', 'Bergstraße 45', '80331',  
'München', '01769876543'),  
...
```

4. Indizes

Um die Performance der Abfragen zu verbessern, wurden verschiedene Indizes erstellt:

```
CREATE INDEX idx_nutzer_email ON nutzer(email);  
CREATE INDEX idx_buch_titel ON buch(titel);
```

...

5. Testabfragen

Beispiel für eine Testabfrage:

Verfügbare Bücher in Berlin

```
SELECT CONCAT(n.vorname,' ',n.nachname) AS Besitzer, b.titel AS Buch, s.stadt  
AS Ort  
FROM buch_exemplar be  
JOIN buch b ON be.buch_id=b.buch_id  
JOIN nutzer n ON be.besitzer_id=n.nutzer_id  
JOIN standort s ON be.standort_id=s.standort_id  
WHERE be.status='verfügbar' AND s.stadt='Berlin';
```

Hinweis: Diese Abfrage listet alle verfügbaren Bücher in Berlin auf, inklusive dem Besitzer und der Stadt.

Fazit

Die Datenbankstruktur ist vollständig und für die Buchtausch-App bereit. Mit der Implementierung der Tabellen, Dummy-Daten und Indizes bietet sie eine solide Grundlage für die App-Entwicklung.